

CI/CD レポート

静的 Web サービスの継続的デプロイ

沼田 陽

2026 年 7 月 12 日

対象リポジトリ	https://github.com/harukiti82/my-static-html-site
公開 URL	https://orange-stone-0cfe86d00.7.azurestaticapps.net

1 CI/CD とは

CI/CD は「継続的インテグレーション (Continuous Integration) / 継続的デリバリー・デプロイ (Continuous Delivery / Deployment)」の略で、ソースコードの変更を**自動的にビルド・テスト・デプロイ**する仕組みを指す。

用語	意味	本レポートでの該当
CI	変更を頻繁に統合し自動で検証する	<code>git push</code> を契機に GitHub Actions が起動
CD	検証を通った成果物を自動で本番へ反映	Azure Static Web Apps へ自動アップロード

従来は「手元でファイルを作る → 手動で FTP アップロード」という流れだったが、CI/CD ではこれを自動化する。本サービスでは **main ブランチへ push した瞬間に、数十秒 ~1 分で世界中に公開される。**

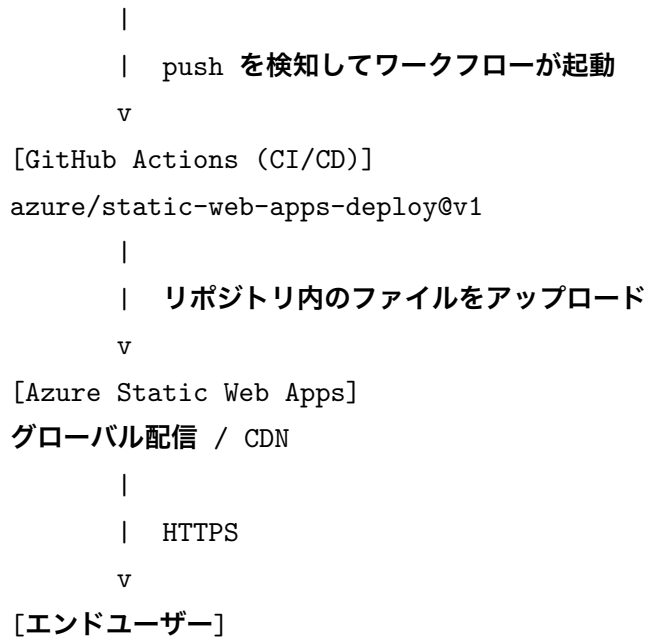
2 システム構成

静的な HTML / CSS のみで構成された Web サイトを、GitHub と Azure を連携させて自動公開している。

[開発者 (ローカル)]

```
|  
| git push origin main  
v
```

[GitHub リポジトリ (main ブランチ)]



使用技術

分類	技術
ソース管理	Git / GitHub
CI/CD 実行基盤	GitHub Actions
デプロイアクション	Azure/static-web-apps-deploy@v1
ホスティング	Azure Static Web Apps (無料プラン)
コンテンツ	静的 HTML (index.html) + CSS (style.css)

3 CI/CD パイプラインの流れ

1. 開発者がローカルで index.html / style.css を編集する
2. git commit して git push origin main する
3. push を検知して **GitHub Actions のワークフローが自動起動**する
4. Azure/static-web-apps-deploy アクションがリポジトリ内のファイルを Azure へアップロードする
5. Azure Static Web Apps がグローバル配信網に反映し、**約 1 分で公開 URL に変更が反映**される

プルリクエスト (PR) を作成した場合は、マージ前でも**プレビュー環境**が自動生成され、本番に影響を与えずに変更内容を確認できる。PR をクローズするとプレビュー環境は自動で破棄される。

4 デプロイ設定（ワークフロー定義）

CI/CD の挙動は `.github/workflows/azure-static-web-apps-orange-stone-0cfe86d00.yml` に定義されている（抜粋）。

```
name: Azure Static Web Apps CI/CD

on:
  push:
    branches:
      - main                # main への push で本番デプロイ
  pull_request:
    types: [opened, synchronize, reopened, closed]
    branches:
      - main                # PR でプレビュー環境を生成 / 破棄

jobs:
  build_and_deploy_job:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3                # (1) ソースを取得
      - name: Build And Deploy
        uses: Azure/static-web-apps-deploy@v1
        with:
          azure_static_web_apps_api_token: ${ secrets.AZURE_..._TOKEN_... }
          action: "upload"
          app_location: "/"                      # (2) 公開するソースの場所
          output_location: "."                  # (3) 成果物ディレクトリ
```

- `on: ...` いつ動くか（トリガー）。ここでは `main` への `push` と `PR`。
- `secrets.AZURE_...TOKEN...` Azure が発行したデプロイ用トークン。GitHub の Secrets に安全に保管され、コードには一切含まれない。
- `app_location / output_location ...` どのフォルダを公開するか。本サイトはビルド不要のため、リポジトリ直下（`/`）をそのまま配信する。

5 ディプロイ画面 (GitHub Actions 実行履歴)

main へ push するたびに、GitHub の **Actions タブ** に実行結果が記録される。以下は `gh run list` で取得した実際のデプロイ実行履歴である。

状態	ワークフロー	トリガー	所要時間	実行日時 (UTC)
成功	Azure Static Web Apps CI/CD	push	1m05s	2026-07-09 16:38
失敗	Azure Static Web Apps CI/CD	push	10m46s	2026-07-09 16:38
成功	Azure Static Web Apps CI/CD	push	1m04s	2026-07-08 01:41
成功	Azure Static Web Apps CI/CD	push	1m04s	2026-07-08 01:13
失敗	Azure Static Web Apps CI/CD	push	43s	2026-07-08 01:10

デプロイ成功時、Actions のログ末尾には公開先 URL が出力される (実際のログ)。

Deployment Complete :)

Visit your site at: <https://orange-stone-0cfe86d00.7.azurestaticapps.net>

スクリーンショット貼付欄 (1) (デプロイ画面)

GitHub リポジトリの **Actions** タブを開き、上記の実行履歴が並んでいる画面を貼り付ける。

URL: <https://github.com/harukiti82/my-static-html-site/actions>

6 変更画面 (コミット履歴と差分)

「いつ・誰が・何を変更したか」は Git のコミット履歴で追跡できる。以下は実際の履歴 (git log)。

```
bc38726 Merge branch 'main' of https://github.com/harukiti82/my-static-html-site
7625825 沼田がしていますを追加
80a8333 沼田を追加
70c1af5 赤色に変更
f94d58f テキストカラーを変更
```

各コミットには具体的な変更差分 (diff) が記録されている。例として 2 つのコミットの差分を示す。

コミット 70c1af5 「赤色に変更」

```
diff --git a/style.css b/style.css
h1 {
```

```
- color: #00ffc3d4;    /* 変更前: 水色 */
+ color: #ff1d1dd4;    /* 変更後: 赤色 */
}
```

コミット 7625825 「沼田がしていますを追加」

```
diff --git a/index.html b/index.html
```

```
- <p>GitHub Actions からデプロイしています。沼田</p>
+ <p>GitHub Actions からデプロイしています。沼田がしています</p>
```

この差分が push された結果、CI/CD が自動起動して**そのまま公開 URL へ反映**される。「文字色を赤に変える」という 1 行の変更が、手作業なしで本番サイトに届く点が CI/CD の効果である。

スクリーンショット貼付欄 (2) (変更画面)

GitHub の **Commits** 画面、または特定コミットの差分（緑＝追加・赤＝削除がハイライトされた画面）を貼り付ける。

URL: <https://github.com/harukiti82/my-static-html-site/commits/main>

7 現在の公開 URL と稼働確認

公開 URL: <https://orange-stone-0cfe86d00.7.azurestaticapps.net>

2026 年 7 月 12 日 時点で稼働を確認済み（HTTP ステータス 200 OK）。配信されている内容は以下の通りで、リポジトリの `index.html` と一致している。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>My Static HTML Site</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Azure Static Web Apps Demo</h1>    <!-- 赤色 (#ff1d1d) で表示 -->
  <p>GitHub Actions からデプロイしています。沼田がしています</p>
</body>
</html>
```

スクリーンショット貼付欄 (3) (現在の URL)

上記 URL をブラウザで開き、アドレスバーの URL と「Azure Static Web Apps Demo」(赤字) が表示された画面を貼り付ける。

注意: ドメインには `.7.` というリージョン識別子が含まれる。`.7.` を省いた `https://orange-stone-0cfe86d00.azurestaticapps.net` では Azure の 404 ページが表示されるため、上記の**完全な URL** でアクセスすること。

8 考察 — 運用で気づいた課題と改善案

実際に運用してみたところ、`push` のたびにワークフローが 1 回失敗していることが分かった(第 5 章の履歴の「失敗」)。

原因

リポジトリ内に**デプロイ用のワークフローファイルが 2 つ**存在している。

- `azure-static-web-apps.yaml` (初期に手動作成したもの)
- `azure-static-web-apps-orange-stone-0cfe86d00.yml` (Azure が自動生成したもの)

両方が `main` への `push` で同時に起動し、**同じ Static Web App へ同時にデプロイしようとして衝突**する。失敗した側のログには次のように記録されていた。

```
Status: InProgress. Time: 581.8(s)
```

```
Upload Timed Out. Unsure if deployment was successful or not.
```

一方が先にデプロイを掴み、もう一方は待たされた末に**アップロードがタイムアウト** (約 10 分) して失敗している。

改善案

不要になった `azure-static-web-apps.yaml` を**削除して 1 本化**すれば、衝突がなくなり、毎回のデプロイが安定して成功するようになる。CI/CD では「1 つの成果物に対してデプロイ経路は 1 本」に保つことが重要だという学びが得られた。

9 まとめ

- CI/CD により、`git push` するだけでビルド～公開までが自動化され、手作業のアップロードが不要になった。
- GitHub Actions + Azure Static Web Apps の組み合わせで、静的サイトを無料か

つ数十秒で世界公開できた。

- **PR プレビュー環境**により、本番に影響を与えず変更を確認できる仕組みも備わっている。
- 一方で、**ワークフローの重複**という運用上の落とし穴も体験し、「デプロイ経路の一本化」の重要性を学んだ。